

HPS/FPGA Based MD5 SoC Design

Ibrahim Ali

Department of Electrical, Computer, and Biomedical Engineering
Toronto Metropolitan University
Toronto, Ontario, Canada
Email: ibrahim1.ali@torontomu.ca

Abstract—This report presents the design, implementation, and evaluation of an HPS/FPGA based MD5 System-on-Chip on the DE1-SoC platform. The completed design integrates a provided 32-engine VHDL MD5 core with custom Avalon memory-mapped slave interfaces and a C application running on the Hard Processor System under Yocto Linux. The HPS software writes 512-bit messages to the FPGA in sixteen 32-bit segments, starts selected engines, polls for completion, reads 128-bit digests back in four 32-bit segments, verifies correctness, and records performance statistics. The final implementation supports both serial and parallel execution modes using the same HPS application. Experimental results show 100% correctness for all measured runs and demonstrate that parallel execution consistently outperforms serial execution, while the Quartus timing and resource reports provide insight into the hardware cost of the implemented accelerator.

Index Terms—MD5, SoC, HPS/FPGA, Avalon-MM, DE1-SoC, hardware acceleration, parallel hashing

I. INTRODUCTION

This project represents the final stage of the HPS/FPGA based MD5 decryption System-on-Chip developed for COE838. The earlier interim report focused on extracting the project requirements, studying the provided MD5 RTL and testbenches, identifying the communication constraints imposed by the lightweight HPS-to-FPGA bridge, and planning the hardware/software partitioning needed for implementation. Building on that earlier work, this final report presents the completed SoC design, the finalized hardware and software implementation, and the measured experimental results.

The objective of the project was to implement a working MD5 hashing system on the DE1-SoC platform using a provided 32-engine VHDL MD5 core together with an HPS software controller. According to the project specification, the final system had to support Avalon memory-mapped communication, segmented transfer of 512-bit messages and 128-bit digests over a 32-bit bridge, control of 32 hashing engines, correctness verification, and performance measurement in both serial and parallel execution modes [1]. The completed design therefore serves as a full hardware/software co-design example in which the planning work from the interim stage has been extended into a functioning and validated SoC prototype.

II. PAST WORK / REVIEW

The work summarized in the interim report formed the direct foundation for the final implementation [2]. At that stage, the main effort was devoted to carefully reviewing the project handout, extracting the major design requirements, and

breaking the project into manageable hardware and software tasks. In particular, the interim work identified the need to understand the provided MD5 RTL hierarchy, determine how messages and digests were transferred, define a register-level communication strategy, and plan the HPS software control flow.

A major part of the interim phase involved studying the supplied RTL and testbench behavior in order to understand how the MD5 algorithm had been mapped into hardware. The project handout specifies that students must determine how an engine is configured, how a 512-bit message is sent to it, how hashing is initiated, how completion is detected, and how the 128-bit digest is read back [1]. This analysis was essential because the final Avalon-MM slave interfaces and HPS software had to match the structure and timing behavior of the supplied MD5 core.

The interim report also established the architectural direction that was eventually used in the final design [2]. A preliminary HPS/FPGA partition was proposed in which the HPS software would communicate with the MD5 engine group through separate Avalon-MM control and data slaves. In addition, a draft register-level communication model and a planned software control sequence were outlined. These early design artifacts served as the basis for the completed implementation presented in this report.

From a theoretical perspective, MD5 processes a 512-bit input block divided into sixteen 32-bit words and generates a 128-bit digest after 64 rounds of operations involving fixed constants, Boolean functions, modular addition, and circular shifts [1]. Because the provided hardware already contains 32 concurrent MD5 engines, the project naturally lends itself to a comparison between serial engine scheduling and parallel engine scheduling under HPS software control.

III. METHODOLOGY

The methodology followed in the final implementation directly extended the staged plan proposed in the interim report. The first stage was the same analytical phase already completed earlier: understanding the provided MD5 core, its testbenches, and the communication constraints of the SoC platform. The second stage converted that analysis into hardware by implementing a wrapper and custom Avalon-MM slave interfaces. The third stage integrated these hardware components into a complete Platform Designer system for the DE1-SoC board. The fourth stage implemented the HPS

controller application in C under Yocto Linux. The final stage validated correctness and collected benchmark, timing, and resource utilization results.

The project specification requires at least two Avalon-MM slaves because the lightweight bridge supports only 32-bit transactions, while the MD5 input message is 512 bits and the digest is 128 bits [1]. This matched the architectural direction already proposed in the interim report. The final design therefore adopted separate control and data slaves, allowing engine start/reset/status operations to be separated from segmented message and digest transfers.

The HPS software methodology also followed the control flow that had been outlined earlier in the interim stage. For a selected engine, software resets the engine, writes the sixteen 32-bit message words, asserts the start bit, waits for the done signal, and reads back the four 32-bit digest words. This sequence was first verified using a single-engine sanity test and was then extended into serial and parallel benchmark modes to evaluate system performance.

A. MD5 Algorithm Invoked in VHDL

The MD5 algorithm processes a 512-bit message block divided into sixteen 32-bit words and produces a 128-bit digest. It uses four internal 32-bit state variables, usually denoted A , B , C , and D , which are initialized to fixed constants. For each of the 64 rounds, the algorithm computes a round-dependent Boolean function, selects a message word index, adds a round constant, and performs a circular left rotation before updating the state variables [1].

The four logical functions commonly used by MD5 are:

$$F(X, Y, Z) = (X \wedge Y) \vee (\neg X \wedge Z) \quad (1)$$

$$G(X, Y, Z) = (D \wedge B) \vee (\neg D \wedge C) \quad (2)$$

$$H(X, Y, Z) = X \oplus Y \oplus Z \quad (3)$$

$$I(X, Y, Z) = Y \oplus (X \vee \neg Z) \quad (4)$$

At the end of the 64 rounds, the updated values are accumulated with the initial state to form the final digest. In the implemented SoC, this algorithm is carried out by the provided MD5 hardware core while the HPS is responsible only for configuration, data movement, verification, and measurement.

IV. DESIGN

A. Overall SoC Architecture

The final HPS/FPGA MD5 system follows the same architectural direction proposed in the interim report. At a high level, the system consists of the Cyclone V HPS, the lightweight HPS-to-FPGA bridge, custom Avalon-MM control and data slaves, a wrapped MD5 hardware block, and the HPS software application. The HPS software communicates with the FPGA by performing 32-bit memory-mapped reads and writes to the custom slave interfaces, while the wrapped hardware block connects those interfaces to the provided 32-engine MD5 core.

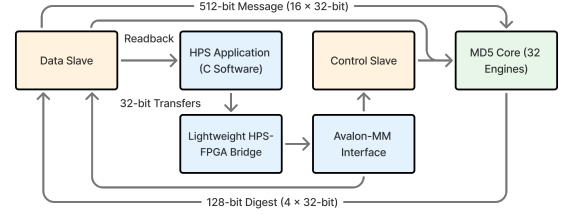


Fig. 1. Conceptual HPS/FPGA MD5 SoC block diagram first developed during the interim design stage.

Figure 1 illustrates the high-level partitioning between the HPS software, the Avalon-MM communication interface, and the 32-engine MD5 hardware core. This block-level organization was retained in the final implementation and later refined into the complete Platform Designer system shown in Fig. 3.

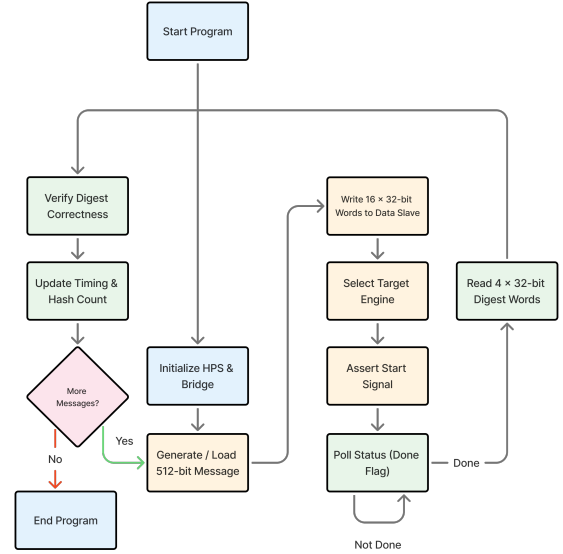


Fig. 2. HPS software control flow for one MD5 hashing transaction.

Figure 2 shows the software execution flow that was initially planned during the interim stage and then implemented in the final HPS controller. The sequence includes message transfer, engine start control, completion polling, digest readback, and correctness verification.

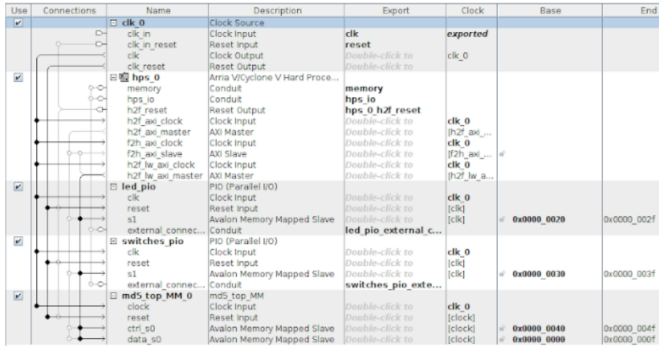


Fig. 3. Final Platform Designer system for the HPS/FPGA MD5 SoC.

As shown in Fig. 3, the final Platform Designer system includes the HPS subsystem, clocking, and the custom MD5 subsystem connected through the lightweight bridge. This satisfies the project requirement that the HPS communicate with Avalon-MM slaves in order to control the MD5 core [1].

B. Custom MD5 Wrapper

The implemented wrapper module, `md5_soc_top`, instantiates three blocks:

- `md5_ctrl_slave`
- `md5_data_slave`
- the provided `md5_group` compute core

This wrapper was used to preserve the original supplied MD5 RTL while localizing all bus-facing behavior in the custom interface modules. The approach simplified debugging and reduced the risk of accidentally modifying the provided accelerator logic.

C. Avalon-MM Control Slave

The control slave provides a simple 32-bit register interface used to control and monitor the engines. The implemented register map is:

- **0x00**: reset register
- **0x04**: start register
- **0x08**: done register
- **0x0C**: status/debug mirror

Each bit in these registers corresponds to one MD5 engine. This allows the HPS to reset or start one engine at a time in serial mode, or all engines at once in parallel mode.

D. Avalon-MM Data Slave

The data slave provides the interface required to transfer message words into the engine memories and retrieve digest words from completed engines. The implemented offsets are:

- **0x00**: write data
- **0x04**: write address
- **0x08**: read address
- **0x0C**: read data

When software writes a 32-bit word to the write-data register and then writes the packed address to the write-address register, the data slave generates the internal write pulse used by the MD5 core. Digest reads are performed by first writing a

packed read address and then reading the selected digest word from the read-data register.

E. Address Mapping

The software uses helper functions to encode the addresses expected by the hardware. For message writes, the packed write address identifies the target engine and the selected word index within the 16-word message. For digest reads, the packed read address identifies the target engine and the selected word index within the four-word digest.

This software-side address packing was derived directly from the RTL study of the supplied core, ensuring that all 32 engines could be addressed correctly without adding extra scheduling logic in hardware.

F. HPS Software Controller

The HPS application was written in C and runs under Yocto Linux. It maps the lightweight bridge into user space using `/dev/mem` and `mmap()`, then accesses the two slave interfaces through base pointers. The application supports three execution modes:

- a single-engine sanity mode
- a serial benchmark mode
- a parallel benchmark mode

In serial mode, one engine is reset, loaded, started, waited on, and read back at a time. In parallel mode, all 32 engines are first loaded, then started together, after which the software polls until the entire done vector becomes all ones. In both cases, the software tracks the total number of hashes, the number of correct hashes, the total execution time, and the overall hash rate.

V. EXPERIMENTAL RESULTS

A. Functional Validation

The first validation step was a single-engine sanity check using a known 16-word message and its expected MD5 digest. The observed digest matched the expected value exactly:

```
baebddf8 61d3eb27 14ba892c 2ad26682
```

in the displayed high-to-low order. This verified correct software-to-hardware communication, correct addressing, correct control sequencing, and correct digest readback.

```
bob@socfpga:~$ launcher md5 sanity
Running single-engine sanity check on engine 0...
Expected digest words: 0xbaebddf8 0x61d3eb27 0x14ba892c 0x2ad26682
Observed digest words: 0xbaebddf8 0x61d3eb27 0x14ba892c 0x2ad26682
Single-engine sanity check PASSED.
```

Fig. 4. Single-engine sanity-check terminal output.

B. Serial and Parallel Benchmark Results

All recorded HPS/FPGA benchmark runs achieved 100% correctness. Table I summarizes the measured results for serial and parallel operation.

TABLE I
HPS/FPGA MD5 BENCHMARK RESULTS

Mode	Iter.	Hashes	Correct	Time (s)	Rate (hashes/s)
Serial	1	32	32	0.000980	32645.400
Serial	10	320	320	0.008383	38172.128
Serial	100	3200	3200	0.080972	39519.932
Serial	500	16000	16000	0.398819	40118.484
Parallel	1	32	32	0.000768	41692.725
Parallel	10	320	320	0.005993	53396.430
Parallel	100	3200	3200	0.060551	52848.248
Parallel	500	16000	16000	0.291240	54937.441

The serial results show that the hash rate increased from 32 645.400 hashes/s at one iteration to 40 118.484 hashes/s at 500 iterations. The parallel results show an increase from 41 692.725 hashes/s to 54 937.441 hashes/s over the same range. This confirms that the parallel execution mode makes better use of the 32-engine hardware.

C. Serial vs Parallel Discussion

To quantify the benefit of parallel execution, the parallel-over-serial speedup was computed as

$$\text{speedup} = \frac{\text{parallel hash rate}}{\text{serial hash rate}}.$$

The measured speedups were approximately:

- $1.277\times$ at 1 iteration
- $1.399\times$ at 10 iterations
- $1.337\times$ at 100 iterations
- $1.369\times$ at 500 iterations

These results show a clear performance advantage for parallel scheduling, but not a $32\times$ speedup. The reason is that the HPS still performs message loading and digest retrieval through a 32-bit memory-mapped interface, so the communication overhead remains significant even though the hashing engines operate concurrently.

D. Quartus Timing Results

The Quartus TimeQuest timing reports for the HPS/FPGA design produced the following values:

- Slow 1100 mV 85°C Fmax: 1184.83 MHz
- Slow 1100 mV 0°C Fmax: 1237.62 MHz

Following the usual interpretation, the lower value was taken as the chosen maximum frequency for discussion purposes.

TABLE II
TIMING PERFORMANCE OF THE HPS/FPGA MD5 SoC

Metric	Value
Slow 1100 mV 85°C Fmax	1184.83 MHz
Slow 1100 mV 0°C Fmax	1237.62 MHz
Chosen Fmax	1184.83 MHz

E. Quartus Resource Utilization

The FPGA resource usage for the implemented HPS/FPGA MD5 SoC is summarized in Table III.

TABLE III
QUARTUS RESOURCE UTILIZATION OF THE HPS/FPGA MD5 SoC

Resource	Usage
ALMs needed	13412
Total LABs used	1936
Combinational ALUTs	14998
Dedicated logic registers	14881
M10K blocks	64
I/O pins	167

These results show that the implemented design occupies a substantial amount of FPGA logic and memory resources, which is expected given that the system contains a 32-engine hashing accelerator together with the required HPS/FPGA interfacing logic.

F. Overall Discussion

The experimental results demonstrate that the final system is functionally correct and that the 32-engine architecture provides real throughput improvement when the engines are scheduled in parallel. Every reported run achieved 100% correctness, which confirms that the message transfer path, control logic, digest retrieval path, and software verification process were implemented properly.

At the same time, the results also show that the full system performance is influenced not only by the hashing hardware itself but also by the software-controlled transfer overhead. Each hash operation requires sixteen message-word writes, start and reset control, polling for completion, and four digest-word reads. Because these steps are performed through a 32-bit bridge, the overall system-level speedup is much smaller than the theoretical engine count might suggest.

VI. CONCLUSION

This final report completes the work that began in the interim stage of the project. The earlier report established the theoretical understanding of the MD5 algorithm, studied the supplied RTL and testbenches, identified the SoC communication constraints, and proposed a practical implementation plan. The present report showed how that planned architecture was fully implemented as a working HPS/FPGA based MD5 System-on-Chip on the DE1-SoC platform.

The completed design successfully integrated the provided 32-engine MD5 VHDL core with custom Avalon memory-mapped control and data slaves and a Linux-based HPS controller application. The final system correctly supported message loading, engine control, digest readback, correctness verification, and benchmark measurement. Experimental results showed 100% correctness in all reported runs and confirmed that parallel execution consistently outperformed serial execution.

REFERENCES

- [1] COE838: Systems-on-Chip Design, “HPS/FPGA Based MD5 Decryption SoC Final Project,” course project handout.
- [2] I. Ali, “COE838 Interim Report,” Toronto Metropolitan University, Mar. 2026.
- [3] Intel Quartus Prime Standard Edition, TimeQuest Timing Analyzer and Fitter reports for the implemented HPS/FPGA MD5 SoC.

A. Platform Designer and Terminal Screenshots

Fig. 5. Final Platform Designer system for the HPS/FPGA MD5 SoC.

Fig. 6. Serial benchmark terminal output for 1 iteration.

Fig. 7. Serial benchmark terminal output for 10 iterations.

Fig. 8. Serial benchmark terminal output for 100 iterations.

Fig. 9. Serial benchmark terminal output for 500 iterations.

Fig. 10. Parallel benchmark terminal output for 1 iteration.

Fig. 11. Parallel benchmark terminal output for 10 iterations.


```

bob@socfpga:~$ launcher md5 parallel 100

=== PARALLEL MODE ===
Iterations: 100
Parallel total hashes   : 3200
Parallel correct hashes : 3200
Parallel execution time : 0.060551 s
Parallel hash rate      : 52848.248 hashes/s
Parallel correctness    : 100.00 %

```

Fig. 12. Parallel benchmark terminal output for 100 iterations.

```

bob@socfpga:~$ launcher md5 parallel 500

=== PARALLEL MODE ===
Iterations: 500
Parallel total hashes   : 16000
Parallel correct hashes : 16000
Parallel execution time : 0.291240 s
Parallel hash rate      : 54937.441 hashes/s
Parallel correctness    : 100.00 %

```

Fig. 13. Parallel benchmark terminal output for 500 iterations.

B. Source Code Listings

Listing 1. md5_soc_top.vhd

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity md5_soc_top is
    port (
        clk           : in  std_logic;
        reset_n       : in  std_logic;
        ctrl_s0_address : in  std_logic_vector(1 downto 0);
        ctrl_s0_write  : in  std_logic;
        ctrl_s0_read   : in  std_logic;
        ctrl_s0_writedata : in std_logic_vector(31 downto 0);
        ctrl_s0_readdata : out std_logic_vector(31 downto 0);
        data_s0_address : in  std_logic_vector(1 downto 0);
        data_s0_write  : in  std_logic;
        data_s0_read   : in  std_logic;
        data_s0_writedata : in std_logic_vector(31 downto 0);
        data_s0_readdata : out std_logic_vector(31 downto 0);
    );
end entity;

architecture rtl of md5_soc_top is
    signal core_reset_bus : std_logic_vector(31 downto 0);
    signal core_start_bus : std_logic_vector(31 downto 0);
    signal core_done_bus  : std_logic_vector(31 downto 0);
    signal core_wr_pulse  : std_logic;
    signal core_write_data : std_logic_vector(31 downto 0);
    signal core_write_addr : std_logic_vector(8 downto 0);
    signal core_read_addr  : std_logic_vector(6 downto 0);
    signal core_read_data  : std_logic_vector(31 downto 0);
begin
    u_ctrl: entity work.md5_ctrl_slave
        port map (
            clk           => clk,
            reset_n       => reset_n,
            ctrl_s0_address => ctrl_s0_address,
            ctrl_s0_write  => ctrl_s0_write,
            ctrl_s0_read   => ctrl_s0_read,
            ctrl_s0_writedata => ctrl_s0_writedata,

```

```

            ctrl_s0_readdata => ctrl_s0_readdata,
            md5_reset       => core_reset_bus,
            md5_start       => core_start_bus,
            md5_done        => core_done_bus
        );

    u_data: entity work.md5_data_slave
        port map (
            clk           => clk,
            reset_n       => reset_n,
            data_s0_address => data_s0_address,
            data_s0_write  => data_s0_write,
            data_s0_read   => data_s0_read,
            data_s0_writedata => data_s0_writedata,
            data_s0_readdata => data_s0_readdata,
            md5_wr        => core_wr_pulse,
            md5_writedata  => core_write_data,
            md5_writeaddr  => core_write_addr,
            md5_readaddr   => core_read_addr,
            md5_readdata   => core_read_data
        );

    u_md5_group: entity work.md5_group
        port map (
            clk           => clk,
            wr            => core_wr_pulse,
            reset         => core_reset_bus,
            start         => core_start_bus,
            writedata     => core_write_data,
            writeaddr     => core_write_addr,
            readaddr      => core_read_addr,
            done          => core_done_bus,
            readdata      => core_read_data
        );
end architecture;

```

Listing 2. md5_ctrl_slave.vhd

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity md5_ctrl_slave is
    port (
        clk           : in  std_logic;
        reset_n       : in  std_logic;
        ctrl_s0_address : in  std_logic_vector(1 downto 0);
        ctrl_s0_write  : in  std_logic;
        ctrl_s0_read   : in  std_logic;
        ctrl_s0_writedata : in std_logic_vector(31 downto 0);
        ctrl_s0_readdata : out std_logic_vector(31 downto 0);
        md5_reset      : out std_logic_vector(31 downto 0);
        md5_start      : out std_logic_vector(31 downto 0);
        md5_done       : in  std_logic_vector(31 downto 0);
    );
end entity;

architecture rtl of md5_ctrl_slave is
    signal reset_hold_reg : std_logic_vector(31 downto 0) := (others => '0');
    signal start_hold_reg : std_logic_vector(31 downto 0) := (others => '0');
begin
    process(clk, reset_n)
    begin
        if reset_n = '0' then
            reset_hold_reg <= (others => '0');
            start_hold_reg <= (others => '0');
        elsif rising_edge(clk) then
            start_hold_reg <= (others => '0');

            if ctrl_s0_write = '1' then
                case ctrl_s0_address is
                    when "00" =>
                        reset_hold_reg <= ctrl_s0_writedata;
                    when "01" =>

```

```

        start_hold_reg <= ctrl_s0_writedata;
    when others =>
        null;
    end case;
end if;
end if;
end process;

process(ctrl_s0_address, ctrl_s0_read, reset_hold_reg,
start_hold_reg, md5_done)
begin
    ctrl_s0_readdata <= (others => '0');

    if ctrl_s0_read = '1' then
        case ctrl_s0_address is
            when "00" =>
                ctrl_s0_readdata <= reset_hold_reg;
            when "01" =>
                ctrl_s0_readdata <= start_hold_reg;
            when "10" =>
                ctrl_s0_readdata <= md5_done;
            when "11" =>
                ctrl_s0_readdata <= md5_done;
            when others =>
                ctrl_s0_readdata <= (others => '0');
        end case;
    end if;
end process;

md5_reset <= reset_hold_reg;
md5_start <= start_hold_reg;
end architecture;

```

Listing 3. md5_data_slave.vhd

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity md5_data_slave is
    port (
        clk                : in  std_logic;
        reset_n            : in  std_logic;
        data_s0_address    : in  std_logic_vector(1 downto 0)
    );
    data_s0_write          : in  std_logic;
    data_s0_read           : in  std_logic;
    data_s0_writedata      : in  std_logic_vector(31 downto
0);
    data_s0_readdata       : out std_logic_vector(31 downto
0);
    md5_wr                 : out std_logic;
    md5_writedata          : out std_logic_vector(31 downto
0);
    md5_writeaddr          : out std_logic_vector(8 downto 0)
    ;
    md5_readaddr           : out std_logic_vector(6 downto 0)
    ;
    md5_readdata           : in  std_logic_vector(31 downto
0)
    );
end entity;

architecture rtl of md5_data_slave is
    signal write_data_reg   : std_logic_vector(31 downto 0)
:= (others => '0');
    signal write_addr_reg   : std_logic_vector(8 downto 0)
:= (others => '0');
    signal read_addr_reg    : std_logic_vector(6 downto 0)
:= (others => '0');
    signal write_strobe_reg : std_logic := '0';
begin
    process(clk, reset_n)
    begin
        if reset_n = '0' then
            write_data_reg <= (others => '0');
            write_addr_reg <= (others => '0');
            read_addr_reg  <= (others => '0');
            write_strobe_reg <= '0';
        elsif rising_edge(clk) then
            write_strobe_reg <= '0';

```

```

        if data_s0_write = '1' then
            case data_s0_address is
                when "00" =>
                    write_data_reg <= data_s0_writedata;
                when "01" =>
                    write_addr_reg  <=
data_s0_writedata(8 downto 0);
                    write_strobe_reg <= '1';
                when "10" =>
                    read_addr_reg <= data_s0_writedata(6
downto 0);
                when others =>
                    null;
            end case;
        end if;
    end process;

    process(data_s0_address, data_s0_read, write_data_reg,
write_addr_reg, read_addr_reg, md5_readdata)
        variable rd_mux_val : std_logic_vector(31 downto 0);
    begin
        rd_mux_val := (others => '0');

        if data_s0_read = '1' then
            case data_s0_address is
                when "00" =>
                    rd_mux_val := write_data_reg;
                when "01" =>
                    rd_mux_val(8 downto 0) := write_addr_reg
;
                when "10" =>
                    rd_mux_val(6 downto 0) := read_addr_reg;
                when "11" =>
                    rd_mux_val := md5_readdata;
                when others =>
                    rd_mux_val := (others => '0');
            end case;
        end if;

        data_s0_readdata <= rd_mux_val;
    end process;

    md5_wr          <= write_strobe_reg;
    md5_writedata    <= write_data_reg;
    md5_writeaddr    <= write_addr_reg;
    md5_readaddr     <= read_addr_reg;
end architecture;

```

Listing 4. main.c

```

#include <errno.h>
#include <fcntl.h>
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <time.h>
#include <unistd.h>

#include "hps_0.h"
#define LW_BRIDGE_BASE 0xFF200000UL
#define LW_BRIDGE_SPAN 0x00200000UL
#define DATA_BASE_OFFSET MD5_TOP_MM_0_DATA_S0_BASE
#define CTRL_BASE_OFFSET MD5_TOP_MM_0_CTRL_S0_BASE
#define CTRL_RESET_REG 0x00
#define CTRL_START_REG 0x04
#define CTRL_DONE_REG 0x08
#define CTRL_STATUS_REG 0x0C
#define DATA_WRITE_DATA 0x00
#define DATA_WRITE_ADDR 0x04
#define DATA_READ_ADDR 0x08
#define DATA_READ_DATA 0x0C
typedef struct {
    volatile uint8_t *lw_bridge_ptr;
    volatile uint32_t *ctrl_regs_ptr;
    volatile uint32_t *data_regs_ptr;
    int mem_handle;
} md5_hw_ctx_t;
static const uint32_t sample_block_words[16] = {

```

```

0x01680208, 0x13ab80bb, 0xcb8b2c30, 0xb9657582, 0
xa3793c48, 0x103f26be,
0x0b78dac4, 0x5c433348, 0x4de99287, 0xe7f0be7c, 0
x00808533, 0x00000000,
0x00000000, 0x00000000, 0x00000150, 0x00000000};
static const uint32_t reference_digest_words[4] = {0
x2ad26682, 0x14ba892c,
0
x61d3eb27, 0xbaebddf8};
static void mmio_write32(volatile uint32_t *base, uint32_t
reg_offset,
uint32_t reg_value) {
base[reg_offset >> 2] = reg_value;
}
static uint32_t mmio_read32(volatile uint32_t *base,
uint32_t reg_offset) {
return base[reg_offset >> 2];
}
static uint32_t build_write_addr(uint32_t engine_idx,
uint32_t msg_word_idx) {
uint32_t cluster_idx;
uint32_t lane_sel;
uint32_t slot_addr;
cluster_idx = engine_idx >> 1;
lane_sel = engine_idx & 0x1;
slot_addr = (lane_sel << 4) | (msg_word_idx & 0xF);
return ((cluster_idx & 0xF) << 5) | (slot_addr & 0x1F);
}
static uint32_t build_read_addr(uint32_t engine_idx,
uint32_t digest_idx) {
return ((engine_idx & 0x1F) << 2) | (digest_idx & 0x3);
}
static int md5_hw_open(md5_hw_ctx_t *ctx) {
memset(ctx, 0, sizeof(*ctx));
ctx->mem_handle = open("/dev/mem", O_RDWR | O_SYNC);
if (ctx->mem_handle < 0) {
perror("open(/dev/mem)");
return -1;
}
ctx->lw_bridge_ptr =
(volatile uint8_t *)mmap(NULL, LW_BRIDGE_SPAN,
PROT_READ | PROT_WRITE,
MAP_SHARED, ctx->mem_handle
, LW_BRIDGE_BASE);
if (ctx->lw_bridge_ptr == MAP_FAILED) {
perror("mmap");
close(ctx->mem_handle);
ctx->mem_handle = -1;
return -1;
}
ctx->ctrl_regs_ptr =
(volatile uint32_t *) (ctx->lw_bridge_ptr +
CTRL_BASE_OFFSET);
ctx->data_regs_ptr =
(volatile uint32_t *) (ctx->lw_bridge_ptr +
DATA_BASE_OFFSET);
return 0;
}
static void md5_hw_close(md5_hw_ctx_t *ctx) {
if (ctx->lw_bridge_ptr && ctx->lw_bridge_ptr !=
MAP_FAILED) {
munmap((void *)ctx->lw_bridge_ptr, LW_BRIDGE_SPAN);
}
if (ctx->mem_handle >= 0) {
close(ctx->mem_handle);
}
memset(ctx, 0, sizeof(*ctx));
ctx->mem_handle = -1;
}
static void md5_issue_reset(md5_hw_ctx_t *ctx, uint32_t
engine_mask) {
mmio_write32(ctx->ctrl_regs_ptr, CTRL_RESET_REG,
engine_mask);
mmio_write32(ctx->ctrl_regs_ptr, CTRL_RESET_REG, 0
x00000000);
}
static void md5_issue_start(md5_hw_ctx_t *ctx, uint32_t
engine_mask) {
mmio_write32(ctx->ctrl_regs_ptr, CTRL_START_REG,
engine_mask);
}
static uint32_t md5_fetch_done_mask(md5_hw_ctx_t *ctx) {
return mmio_read32(ctx->ctrl_regs_ptr, CTRL_DONE_REG);
}

```

```

}
static void md5_push_message(md5_hw_ctx_t *ctx, uint32_t
engine_idx,
const uint32_t block_words[16])
{
uint32_t i;
for (i = 0; i < 16; i++) {
mmio_write32(ctx->data_regs_ptr, DATA_WRITE_DATA,
block_words[i]);
mmio_write32(ctx->data_regs_ptr, DATA_WRITE_ADDR,
build_write_addr(engine_idx, i));
}
}
static void md5_pull_digest(md5_hw_ctx_t *ctx, uint32_t
engine_idx,
uint32_t digest_buf[4]) {
uint32_t i;
for (i = 0; i < 4; i++) {
mmio_write32(ctx->data_regs_ptr, DATA_READ_ADDR,
build_read_addr(engine_idx, i));
digest_buf[i] = mmio_read32(ctx->data_regs_ptr,
DATA_READ_DATA);
}
}
static bool digest_is_equal(const uint32_t actual_digest[4],
const uint32_t golden_digest[4])
{
int i;
for (i = 0; i < 4; i++) {
if (actual_digest[i] != golden_digest[i]) return
false;
}
return true;
}
static void print_digest_vector(const uint32_t digest_buf
[4]) {
printf("0x%08x 0x%08x 0x%08x 0x%08x\n", digest_buf[3],
digest_buf[2],
digest_buf[1], digest_buf[0]);
}
static double timestamp_seconds(void) {
struct timespec monotonic_ts;
clock_gettime(CLOCK_MONOTONIC, &monotonic_ts);
return (double)monotonic_ts.tv_sec + 1e-9 * (double)
monotonic_ts.tv_nsec;
}
static int run_one_engine_sanity(md5_hw_ctx_t *ctx) {
uint32_t digest_buf[4];
printf("Running single-engine_idx sanity check on engine_idx
0...\n");
md5_issue_reset(ctx, 0x00000001);
md5_push_message(ctx, 0, sample_block_words);
md5_issue_start(ctx, 0x00000001);
while ((md5_fetch_done_mask(ctx) & 0x00000001) == 0) {}
md5_pull_digest(ctx, 0, digest_buf);
printf("Expected digest_buf words: ");
print_digest_vector(reference_digest_words);
printf("Observed digest_buf words: ");
print_digest_vector(digest_buf);
if (!digest_is_equal(digest_buf, reference_digest_words)) {
printf("ERROR: single-engine_idx digest_buf mismatch
.\n");
return -1;
}
}
printf("Single-engine_idx sanity check PASSED.\n");
return 0;
}
static void run_serial_mode(md5_hw_ctx_t *ctx, uint32_t
loop_count) {
uint64_t hash_total = 0, hash_correct = 0;
double time_start, time_stop, elapsed_sec;
uint32_t loop_idx, engine_idx;
time_start = timestamp_seconds();
printf("\n=== SERIAL MODE ===\n");
printf("Iterations: %u\n", loop_count);
for (loop_idx = 0; loop_idx < loop_count; loop_idx++) {
for (engine_idx = 0; engine_idx < 32; engine_idx++)
{
uint32_t digest_buf[4];
md5_issue_reset(ctx, (1u << engine_idx));
md5_push_message(ctx, engine_idx,
sample_block_words);
md5_issue_start(ctx, (1u << engine_idx));
}
}
}

```



```

        while ((md5_fetch_done_mask(ctx) & (1u <<
engine_idx)) == 0) {
            md5_pull_digest(ctx, engine_idx, digest_buf);
            hash_total++;
            if (digest_is_equal(digest_buf,
reference_digest_words)) {
                hash_correct++;
            }
        }
    }
    time_stop = timestamp_seconds();
    elapsed_sec = time_stop - time_start;
    printf("Serial total hashes : %llu\n", (unsigned long
long)hash_total);
    printf("Serial correct hashes : %llu\n", (unsigned long
long)hash_correct);
    printf("Serial execution time : %.6f s\n", elapsed_sec);
    printf("Serial hash rate : %.3f hashes/s\n",
(double)hash_total / elapsed_sec);
    printf(
        "Serial correctness : %.2f %%\n",
        hash_total ? 100.0 * ((double)hash_correct / (double
)hash_total) : 0.0);
}
static void run_parallel_mode(md5_hw_ctx_t *ctx, uint32_t
loop_count) {
    uint64_t hash_total = 0, hash_correct = 0;
    double time_start, time_stop, elapsed_sec;
    uint32_t loop_idx, engine_idx;
    time_start = timestamp_seconds();
    printf("\n=== PARALLEL MODE ===\n");
    printf("Iterations: %u\n", loop_count);
    for (loop_idx = 0; loop_idx < loop_count; loop_idx++) {
        uint32_t digest_buf[4];
        uint32_t finished_mask;
        md5_issue_reset(ctx, 0xFFFFFFFF);
        for (engine_idx = 0; engine_idx < 32; engine_idx++)
        {
            md5_push_message(ctx, engine_idx,
sample_block_words);
        }
        md5_issue_start(ctx, 0xFFFFFFFF);
        do {
            finished_mask = md5_fetch_done_mask(ctx);
        } while (finished_mask != 0xFFFFFFFF);
        for (engine_idx = 0; engine_idx < 32; engine_idx++)
        {
            md5_pull_digest(ctx, engine_idx, digest_buf);
            hash_total++;
            if (digest_is_equal(digest_buf,
reference_digest_words)) {
                hash_correct++;
            }
        }
    }
    time_stop = timestamp_seconds();
    elapsed_sec = time_stop - time_start;
    printf("Parallel total hashes : %llu\n", (unsigned long
long)hash_total);
    printf("Parallel correct hashes : %llu\n",
(unsigned long long)hash_correct);
    printf("Parallel execution time : %.6f s\n", elapsed_sec
);
    printf("Parallel hash rate : %.3f hashes/s\n",
(double)hash_total / elapsed_sec);
    printf(
        "Parallel correctness : %.2f %%\n",
        hash_total ? 100.0 * ((double)hash_correct / (double
)hash_total) : 0.0);
}
static void print_usage(const char *exe_name) {
    fprintf(stderr,
        "Usage:\n %s sanity\n %s serial <
loop_count >\n %s parallel<loop_count>\n ",
        exe_name,
        exe_name, exe_name);
}
int main(int argc, char **argv) {
    md5_hw_ctx_t ctx;
    if (argc < 2) {
        print_usage(argv[0]);
        return 1;

```

```

    }
    if (md5_hw_open(&ctx) != 0) {
        return 1;
    }
    if (strcmp(argv[1], "sanity") == 0) {
        int status_code = run_one_engine_sanity(&ctx);
        md5_hw_close(&ctx);
        return (status_code == 0) ? 0 : 1;
    }
    if (argc < 3) {
        print_usage(argv[0]);
        md5_hw_close(&ctx);
        return 1;
    }
    {
        uint32_t loop_count = (uint32_t)strtoul(argv[2],
NULL, 10);
        if (loop_count == 0) {
            fprintf(stderr, "loop_count must be > 0\n");
            md5_hw_close(&ctx);
            return 1;
        }
        if (strcmp(argv[1], "serial") == 0) {
            run_serial_mode(&ctx, loop_count);
        } else if (strcmp(argv[1], "parallel") == 0) {
            run_parallel_mode(&ctx, loop_count);
        } else {
            print_usage(argv[0]);
            md5_hw_close(&ctx);
            return 1;
        }
    }
    md5_hw_close(&ctx);
    return 0;
}

```